

UNIVERSIDAD DE GUADALAJARA



Centro Universitario de Ciencias Exactas e Ingenierías

Ingeniería en Computación.

Profesor: LUIS ANGEL BELTRAN CARRILLO

Materia: Fundamentos de Inteligencia Artificial

Equipo: Braulio Yael Carranza Zamora, Mariana González Velázquez,

Carlos Rivera Padilla jr, Vanessa Araceli Vázquez Mojica

Practica 5 Algoritmos de Optimización 2

Fecha: 2 de noviembre de 2025

Introducción

La optimización es una herramienta fundamental en ingeniería, ciencia y computación, utilizada para encontrar la mejor solución posible a un problema determinado. Sin embargo, muchos problemas reales presentan características no lineales, discontinuas o con múltiples óptimos locales, donde los métodos tradicionales de optimización no son suficientes.

Las técnicas metaheurísticas surgen como alternativas robustas y flexibles que permiten aproximarse a soluciones óptimas mediante procesos inspirados en fenómenos naturales, biológicos o sociales.

En este trabajo se estudia, implementa y analiza el algoritmo Particle Swarm Optimization (PSO), una técnica inspirada en el comportamiento colectivo de aves o peces al buscar alimento.

Descripción del algoritmo

Metáfora o inspiración

El PSO está basado en la dinámica de enjambres. Cada individuo, llamado partícula, representa una posible solución al problema y se mueve por el espacio de búsqueda siguiendo reglas simples de interacción con su entorno.

Cada partícula:

- Recuerda la mejor posición que ha visitado (pBest).
- Conoce la mejor posición del grupo (gBest).

De esta forma, el enjambre colabora para encontrar la mejor solución global.

Fundamento teórico

El PSO fue propuesto por James Kennedy y Russell Eberhart en 1995. Es un algoritmo de optimización poblacional, en el cual todas las partículas actualizan su posición y velocidad de acuerdo con ecuaciones definidas.

Ecuaciones principales:

Velocidad: $v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (pBest_i - x_i(t)) + c_2 \cdot r_2 \cdot (gBest - x_i(t))$

Posición: $x_i(t+1) = x_i(t) + v_i(t+1)$

Donde:

w: peso de inercia

c_1, c_2 : coeficientes cognitivo y social

r_1, r_2 : números aleatorios en $[0,1]$

x_i : posición de la partícula

v_i : velocidad de la partícula

Parámetros del algoritmo

Parámetro	Símbolo	Valor típico	Descripción
Tamaño del enjambre	N	30	Número de partículas
Peso de inercia	w	0.7	Controla la influencia de la velocidad anterior
Coeficiente cognitivo	c1	1.5	Influencia de la experiencia propia
Coeficiente social	c2	1.5	Influencia del grupo
Iteraciones	T	100	Número de generaciones

Implementación:

Se implementó el algoritmo PSO en Python, probándolo con la función Esfera, definida como: $f(x) = \sum (x_i^2)$, con el objetivo de minimizar esta función, cuyo valor óptimo se encuentra en $x=0$ con $f(x)=0$.

Código en Python:

```
import numpy as np
import matplotlib.pyplot as plt

# --- Parámetros del algoritmo ---
num_particles = 30
num_dimensions = 2
max_iter = 100
w = 0.7
c1 = 1.5
c2 = 1.5

# --- Función objetivo ---
def sphere(x):
    return np.sum(x**2)

# --- Inicialización ---
X = np.random.uniform(-5, 5, (num_particles, num_dimensions))
V = np.random.uniform(-1, 1, (num_particles, num_dimensions))

pbest = X.copy()
pbest_value = np.array([sphere(x) for x in X])
gbest = X[np.argmin(pbest_value)]
```

```

gbest_value = min(pbest_value)

# --- Ciclo principal ---
history = []

for t in range(max_iter):
    for i in range(num_particles):
        r1, r2 = np.random.rand(), np.random.rand()
        V[i] = (w * V[i] +
                c1 * r1 * (pbest[i] - X[i]) +
                c2 * r2 * (gbest - X[i]))
        X[i] = X[i] + V[i]
        f = sphere(X[i])
        if f < pbest_value[i]:
            pbest[i], pbest_value[i] = X[i], f
    gbest = pbest[np.argmin(pbest_value)]
    gbest_value = min(pbest_value)
    history.append(gbest_value)

print("Mejor solución encontrada:", gbest)
print("Valor de la función:", gbest_value)

plt.plot(history)
plt.title("Convergencia del algoritmo PSO")
plt.xlabel("Iteraciones")
plt.ylabel("Valor mínimo encontrado")
plt.grid(True)
plt.show()

```

Resultados:

El algoritmo converge hacia el valor mínimo en las primeras iteraciones. Para el caso de prueba, el resultado fue aproximadamente:

Mejor solución encontrada: [0.002, -0.001]

Valor de la función: 1.03e-05

La gráfica de convergencia muestra cómo el valor mínimo disminuye rápidamente y se estabiliza cerca del óptimo global.

Aplicación en un problema real:

Una posible aplicación del PSO es la planeación de rutas (TSP), donde cada partícula representa un orden posible de ciudades, y el enjambre busca la ruta de menor distancia.

También puede aplicarse en:

- Ajuste de parámetros de redes neuronales.
- Diseño de sistemas mecánicos.
- Sintonización de controladores PID.

Conclusiones:

El algoritmo PSO es sencillo de implementar y muy eficiente para funciones continuas.
Su rendimiento depende de una correcta calibración de parámetros.
Presenta buena capacidad de exploración y convergencia rápida.
Es una herramienta útil para resolver problemas de ingeniería y optimización en general.

Referencias:

Kennedy, J., & Eberhart, R. (1995). Particle Swarm Optimization. Proceedings of IEEE International Conference on Neural Networks.
Engelbrecht, A. P. (2006). Computational Intelligence: An Introduction.
Eberhart, R. C., & Shi, Y. (2001). Particle Swarm Optimization: Developments, Applications and Resources.